FEASIBILITY & INTEGRATION ASSESSMENT

Incorporating the Web-Accessibility POC & Phase 2 into the AI-First SDLC Platform

A grounded assessment — based on a direct study of the platform's source, docs, and agent definitions — of whether the accessibility QA POC and Phase 2 can be built into the existing multi-tenant, engine-agnostic SDLC platform, whether Claude Code can build it, and how much is new vs. reusable.

Verdict in one line: Yes — it's a strong fit and mostly an *extension* of what already exists (the testing agent already runs axe-core WCAG scans, there's already a WCAG gate and a defect-loopback auto-fix loop). Claude Code can build it. No architectural rebuild is required.

Prepared by: Senior AI/ML Engineering Team, Edvensoft
Studied: testing agent (09), test-automation agent (09b), Master Build Blueprint, AGENTS.md, migration spec
Date: August 2026 • **Classification:** Confidential

1. What we studied, and the headline finding

Before answering, we read the platform's actual source of truth in the connected repo — not assumptions. The findings below are cited to specific agents, docs, and locked decisions.

1.1 Evidence base (files studied in the repo)

- **Testing agent** — `agents/09_testing/agent.md` (read in full, as requested). Already runs **axe-core against WCAG 2.1 AA** per screen (Step 9), manual keyboard checks, WCAG→severity mapping, and opens defects via loopback.
- **Test-automation agent** — `agents/09b_test_automation/agent.md`. Already uses **Playwright MCP** to navigate the live app, take **accessibility-tree snapshots**, and generate Page-Object test scripts in many frameworks.
- **Master Build Blueprint** — `docs/SDLC_Platform_Master_Build_Blueprint.md`. Confirms engine-agnostic `ExecutionDriver` (Claude default), capability-not-vendor abstractions, Artifact Registry, Traceability Graph, gates-as-Approval-objects, and — critically — concern **PG21: "a11y (WCAG) gate"** already a locked, built feature (Phase 3), plus **PG11 defect loopback** and an eval/quality harness (PG24).
- Supporting: `AGENTS.md`, `docs/AGENT_MIGRATION_SPEC.md`, `docs/PHASE_2_BUILD_SPEC.md`, `CLAUDE.md`.

1.2 The headline finding

> The platform is **already an accessibility-testing platform in miniature.** It scans for WCAG (2.1 AA) with axe-core, gates on a WCAG a11y gate, emits `Accessibility`-type test cases, maps WCAG levels to defect severity, and auto-opens/-closes a bug per failing check through the defect loopback (which *is* the remediation loop). Our POC's entire **rule layer, shift-left delivery, human-gating, WCAG-criterion mapping, and auto-fix loop already exist**. What the POC and Phase 2 *add* is the **AI semantic/visual layer, a WCAG 2.2 uplift, grounding/confidence, and agentic interaction testing** — net-new, but bounded, and they slot onto existing seams.



Rough engineering split (detailed in §4): most of the plumbing is reused or extended; the genuinely new work is the AI checker capability, the WCAG knowledge base, and the agentic testing layer.

2. Q1 — Can we incorporate the POC & Phase 2? Pros & cons

Answer: Yes, and it's an unusually clean fit — because an accessibility QA capability is exactly the kind of thing this platform was built to host: a gated, traceable, tenant-scoped capability driven by a phase agent.

2.1 How the POC/Phase-2 pieces map onto existing platform components

POC / Phase-2 piece (from Report 1 & Addendums)	Platform home	Status
Rule engine (axe-core, WCAG scan)	Testing agent (09) Step 9	**ALREADY EXISTS** — axe-core WCAG 2.1 AA per screen
Shift-left delivery (catch before merge)	The SDLC phases themselves: UX (05), implementation (07), code review (08), testing (09)	**ALREADY EXISTS** — the whole platform *is* shift-left
WCAG gate / merge gate	a11y (WCAG) gate (PG21) + gates-as-Approval	**ALREADY EXISTS**
Findings cited to WCAG criteria + severity	Testing agent severity table (WCAG A→Critical, AA→High)	**ALREADY EXISTS**
Auto-fix / remediation loop	Defect loopback (PG11) → implementation agent (07) fixes → re-test	**ALREADY EXISTS** — this is the "self-healing" loop
Rendered capture + accessibility tree	Test-automation agent (09b) via Playwright MCP snapshots	**ALREADY EXISTS**
Human-in-the-loop review of findings	Review gates (QA Lead), Approval objects	**ALREADY EXISTS**
Traceability (finding ↔ requirement ↔ screen)	Traceability Graph + `test_case` / `test_report` artifacts	**ALREADY EXISTS**
Cost metering / eval harness / audit	Observability plane (PG19/PG24), Run History	**EXISTS** (Phase 5 depth)
AI semantic/visual checkers (VLM) — meaningful alt, colour-only, fake semantics, labels	New capability behind the testing agent	**NET-NEW**
WCAG 2.2 uplift (from 2.1)	Testing agent + a11y gate + UX agent content	**EXTEND (content)**

Grounding (WCAG KB/RAG) + confidence gating	Knowledge Index (pgvector already in stack) + gate policy	NET-NEW (small)
Phase-2 agentic AT-emulation (keyboard/journey)	Extends 09b Playwright-MCP harness + E2E	NET-NEW

2.2 Pros & cons of incorporating it

Pros

- **Massive reuse:** the rule layer, WCAG gate, severity mapping, defect/auto-fix loop, Playwright capture, review gates, and traceability are already built — the POC lands on existing seams.
- **Product differentiation:** turns a generic axe-core scan into a *SOTA, AI-assisted, WCAG-2.2* accessibility capability spanning design→build→test — a sellable enterprise feature.
- **Gated, tenant-scoped, audited by default:** the platform's RLS, gates, Run History, and cost metering give the AI findings the governance a compliance context needs — and neutralise the over-claim/legal risk (human sign-off is already mandatory).
- **Engine-agnostic:** the AI checks can be a capability behind the driver, so they don't lock the platform to one model.
- **Auto-remediation for free:** the existing defect loopback means an AI-found a11y issue already flows to the implementation agent for a fix and re-test — the "self-healing" feature is largely present.

Cons / risks

- **VLM cost & latency at multi-tenant scale:** per-screen VLM calls add cost/time to the testing phase — needs diff-scoping + caching + the cost-metering plane (Phase 5).
- **Non-determinism vs. the platform's reproducibility ethos:** LLM/VLM output must be confidence-gated, version-pinned, and human-reviewed to fit the gate model (mitigated by existing gates + eval harness).
- **Roadmap contention:** the platform is mid-build (active Phases 4.7/4.8); adding scope competes for capacity — best slotted as an enhancement to the already-planned a11y gate / testing agent, not a new phase.
- **Agentic reliability (Phase 2):** keyboard/journey AT-emulation is the genuinely hard part and needs iteration + the eval harness before it can gate a merge.
- **Standard drift:** WCAG 2.1→2.2 content uplift must be applied consistently across the testing agent, gate, and UX agent.

> **Net:** the pros are structural (you reuse a lot and gain a differentiator) and the cons are the ordinary engineering concerns the platform already has machinery for (cost metering, gates, eval harness). There is no architectural blocker.

3. Q2 — Is Claude Code capable enough to build the full POC & Phase 2?

Answer: Yes — to build it, and to power the AI checks at runtime — with a few honest engineering caveats. Claude Code is already the platform's default engine, building UI screens and reading them during implementation; the accessibility work is the same class of task.

3.1 Why Claude Code is capable

- **It is agentic with tool-use:** it already calls MCP tools, drives **Playwright MCP**, reads the accessibility tree, and writes code into the worktree — exactly what building and running the a11y checkers needs.
- **Claude is itself a VLM:** the models behind Claude Code are multimodal, so Claude can *see* screenshots — it can serve as the vision model for the two vision-dependent checkers (meaningful alt, colour-only) as well as the text checkers, per Addendum B. No separate model is strictly required to prove the POC.
- **It builds the components:** the AI checker service, the new MCP capability, the WCAG knowledge base, the agent-body updates, and the Playwright scripts are all ordinary build tasks Claude Code already performs.
- **The remediation loop already fits it:** Claude Code (as the implementation agent) is already the thing that fixes defects the loopback opens — including a11y defects.

3.2 The honest caveats (what "Claude Code builds it" does and doesn't mean)

Caveat	What it means for the build
Some pieces are **libraries, not model skills**	Classical CV flash-analysis (PEAT-style, SC 2.3.1), ASR (Whisper, captions), and OCR (images-of-text) are components Claude Code *integrates*, not things the model "is." Claude Code writes the glue; the libraries do the work.
Keep the checker **behind the capability/driver seam**	Per the platform's own "capabilities-not-vendors" and engine-agnostic rules, the runtime a11y-AI check should be a capability (e.g. `a11yai.check_*`) — so it isn't hard-wired to Claude and can later route to a self-hosted VLM. Claude Code *builds* it; the platform *runs* it swappably.
Agentic reliability is the hard 20%	Claude Code can build the agentic keyboard/journey tester, but getting it reliable enough to gate a merge needs iteration against the **eval harness** (already planned, PG24). Treat Phase 2 as "build then earn the gate," not "ship gating on day one."
Determinism & cost are engineering, not model, problems	Temperature-0, version pinning, caching, diff-scoping, and confidence gating are things Claude Code implements; they're needed so VLM output fits the platform's reproducible-gate model at tenant scale.

> **Bottom line for Q2:** Claude Code is capable of building the complete POC and Phase 2, and Claude-as-VLM can power the semantic/visual checks for the POC without any other model. The caveats are about *engineering discipline* (put it behind the capability seam, integrate CV/ASR/OCR libraries, harden agentic reliability against the eval harness) — not about a capability gap in Claude Code.

4. Q3 — Huge/drastic changes, or extend what exists?

Answer: Overwhelmingly extend — no drastic rebuild. The platform's architecture (phase agents + capabilities + gates + defect loopback + AgentSpec registry) is exactly the shape that lets you add an AI a11y capability by extension. Below is the concrete change list.

4.1 What you reuse as-is

Rendering + accessibility-tree capture (09b Playwright MCP); axe-core rule scanning (09 Step 9); the a11y WCAG gate; WCAG→severity mapping; `Accessibility` `test_case` + `test_report` artifacts; the defect loopback (auto-open/-close bugs) as the remediation loop; review gates; traceability; Run History/audit; cost metering; the eval harness. **No change needed**

4.2 What you extend (existing files, modest edits)

Component	Extension
Testing agent `09_testing/agent.md` Step 9	Add the **AI semantic/visual checks** (call the new `a11yai` capability) alongside axe-core; uplift **WCAG 2.1 → 2.2**; emit AI findings with confidence + cited criterion into the existing `test_report` . This is a *15%-style* data/reference edit per the migration spec — the reasoning stays.
Test-automation agent `09b`	Add generation of **AI-check and agentic a11y scripts** next to the existing Playwright scripts (it already has the Playwright-MCP harness).
a11y (WCAG) gate criteria	Extend the gate's pass/fail inputs from "axe-only" to "axe + AI-confirmed," at WCAG 2.2.
UX design agent `05` (optional, further shift-left)	Add design-time AI a11y checks against the Design-System artifact — the platform already has a Design-System artifact + a11y gate.
Code-review agent `08` (optional)	Add an a11y dimension so semantic issues are caught at review, before test.

4.3 What is genuinely net-new (the ~30%)

New component	What it is
a11y-AI capability + MCP tool	The grounded VLM/LLM checker service (meaningful-alt, fake-semantics, colour-only, label) exposed as a capability (`a11yai.check_*`) behind the driver — the one substantial new build.
WCAG 2.2 + ACT knowledge base (RAG)	Ingest WCAG 2.2/ACT into the *existing* pgvector Knowledge Index for grounding — small, reuses infra.
Confidence & guardrail layer	Thresholds + "uncertain→human" routing feeding the existing review gate.
Agentic a11y tester (Phase 2)	Keyboard/journey/cross-page emulation on top of the 09b Playwright-MCP harness — the larger Phase-2 build.
Supporting for the human-gated tier	Whisper (caption checks), PEAT-style CV (flash), OCR (images-of-text) — library integrations.

> **Verdict for Q3: ~70% reuse/extend, ~30% net-new; zero architectural rebuild.** The single meaningful new build is the grounded AI-checker capability; everything else is either already present (rule scan, gate,

> loopback, capture, traceability) or a bounded content/extension edit to existing agents. This is precisely the kind of additive capability the platform’s plane/capability/agent architecture is designed to absorb.

5. Using the existing testing agent for the POC & Phase 2

As requested, here is precisely how the current testing agent (which already emits WCAG 2.1 cases) is reused — and the exact points it is extended.

5.1 What it already gives us (reuse verbatim)

- **Step 9 Accessibility Testing:** runs `axe-core` via Playwright against every screen in the wireframe screen-inventory; gates on zero Critical (WCAG A) and zero Serious (WCAG AA) violations; logs Moderate/Minor for tracking.
- **Manual keyboard checklist:** tab order, visible focus, focus trap in modals, screen-reader announcement of errors — the human-required checks, already scripted as a checklist.
- **Severity mapping:** WCAG Level A violation → Critical/High defect; AA → High/Medium — already consistent with our Addendum A ownership model.
- **Artifacts & loopback:** records violations in `test_report`; the platform opens a `defect` per failing check and closes it when it passes — the remediation loop.
- **Automation handoff:** marks accessibility cases as automation candidates for 09b.

5.2 The exact extension points (the edits)

Where in the agent	Edit
Step 9, “Automated scan”	After the axe-core run, add an **AI-checks pass**: for each screen’s candidate elements, call `a11yai.check_*` (meaningful alt / fake semantics / colour-only / label) and merge confidence-scored, criterion-cited findings into the same report.
Standard reference (Step 9 header, Evidence Base, gate)	Change **“WCAG 2.1 AA”** → **“WCAG 2.2 AA”** consistently (the 2.2 uplift).
Manual keyboard checklist	In Phase 2, replace/augment the manual checklist with the **agentic AT-emulation** results (keyboard journeys, focus order) — still human-gated.
Severity table & report format	Add a **confidence** column and an “AI / rule / human” source column so AI findings are transparent and low-confidence ones route to review, not to a hard fail.

Per the platform’s own migration spec (75/15/10), these are “~15% data-reference” edits plus one new capability call — the agent’s reasoning, structure, and gates are preserved. It is an *upgrade* of a working agent, not a rewrite.

5.3 Recommended sequencing (fits the current roadmap)

1. **POC (now):** build the `a11yai` capability + WCAG-2.2 KB; wire the AI-checks pass into testing agent Step 9; extend the a11y gate to “axe + AI-confirmed.” Prove it on one tenant/app against a golden set (per Report 1 §8). Reuses everything else.
2. **Shift-left (fast follow):** add the AI a11y checks to the UX (05) and code-review (08) agents so issues are caught earlier — small edits, big value.
3. **Phase 2:** build the agentic AT-emulation tester on the 09b Playwright-MCP harness; harden against the eval harness before letting it gate; add Whisper/PEAT/OCR for the human-gated tier.

Recommendation

> Proceed. Incorporate the POC as an **enhancement to the existing testing agent + a11y gate** (not a new phase), build the one net-new `a11yai` capability behind the driver seam, and let the existing defect-loopback, gates, traceability, and cost/eval planes do the rest. Claude Code can build and power it. Expect **~70% reuse, ~30% net-new, and no architectural change** — with Phase 2's agentic layer as the one part that needs reliability iteration before it earns a gating role.

Basis & caveats: grounded in a direct read of `agents/09_testing/agent.md`, `agents/09b_test_automation/agent.md`, and `docs/SDLC_Platform_Master_Build_Blueprint.md` (concerns PG11, PG21, PG24; locked decisions on ExecutionDriver, capabilities, gates). The reuse/new split (~70/30) is an engineering estimate that firms up after a scoping spike on the `a11yai` capability. WCAG criterion coverage, the Rule/AI/Human split, and the tech stack are detailed in Report 1 and Addendums A & B.